

REVISION 2

DOCKER STANDALONE

Self-Hosted Data Pipeline Site — Docker Deployment Guide

A step-by-step reference for deploying, verifying, and troubleshooting a Docker-based standalone Singularity Data Pipeline site.

Overview

A self-hosted Singularity Data Pipeline site can run as either a Kubernetes/K3s deployment or a **Docker standalone site** — a single container (`observo-standalone-site`) that runs every Data Pipeline service under an s6 supervisor (`control-plane-init` , `dataplane` , `control-agent` , `collector` , `pattern-extractor` , `monitor`). This guide covers the Docker standalone deployment model.

Networking and firewall requirements

A self-hosted site needs outbound **HTTPS (TCP 443)** to the SentinelOne cloud — no inbound connections are required for the control plane. The exact hostnames are region-specific (e.g. `na1` , `eu1` , `apac1`):

Endpoint	Purpose
<code>datapipeline.<region>.sentinelone.net</code>	Manager / auth control plane
<code>datapipeline-api.<region>.sentinelone.net</code>	API gateway
<code>datapipeline-logs.<region>.sentinelone.net</code>	Data ingestion
<code>datapipeline-metrics.<region>.sentinelone.net</code>	Metrics
Container registry (as provisioned)	Image pulls

If sources will be published on additional container ports (e.g. Syslog TCP/UDP/TLS listeners), **inbound** access to those specific host ports is also required from whatever is sending data to the site.

NOTE

If your environment uses a corporate proxy, `HTTPS_PROXY` / `HTTP_PROXY` must be set on the container so it can reach the endpoints above.

Prerequisites

Before deploying a self-hosted site using Docker, ensure the following:

Infrastructure

- A Linux host with Docker installed
- Minimum resources: 2 vCPU, 4 GB RAM, 20 GB disk
- Recommended: 4 vCPU, 8 GB RAM, 50 GB disk (supports ~1–2 TB/day throughput)

Network access

- Outbound HTTPS (TCP 443) to SentinelOne cloud endpoints (see Networking and firewall requirements above)
- If using a corporate proxy, ability to configure `HTTPS_PROXY` and `HTTP_PROXY` environment variables

Permissions

- Modify access to the SentinelOne scope where the site will be created
- Ability to run Docker containers on the target host
- Root or sudo access to run Docker commands

Deploy a Docker-based self-hosted site

1 Create the site in Data Pipelines

1. Sign in to your SentinelOne console.
2. Click **Data Pipelines**.
3. Go to **Pipeline Manager > Settings**.
4. In **Sites**, click **Add Site**.
5. Select **Self-hosted site** and click **Next**.
6. Enter a **Name** for the site (for example, "Production Syslog Collector").
7. Enter a **Description** (optional).
8. In the **Environment** dropdown, select **Docker**.
9. Select the **Scope** where this site will collect data. You can only select scopes where you have modify access.
10. Click **Next**.
11. Review the configuration and click **Done**.

The site is created and the Setup Instructions panel appears.

2 Download the configuration file

1. In the Setup Instructions panel, click **Download .env file**.
2. Save the file to your deployment host.

The `.env` file contains:

- Site ID
- Manager endpoint URL
- Authentication token
- Organization ID
- Docker registry credentials

3 Install Data Pipeline Doctor on the Linux host

Data Pipeline Doctor is the tool used both to stand up the Docker site and, afterward, to diagnose it. Install `git` and clone the repo on the Linux host that will run the container.

```
# Debian / Ubuntu
sudo apt-get update && sudo apt-get install -y git

# RHEL / CentOS Stream / Rocky / AlmaLinux
sudo dnf install -y git

# Fedora
sudo dnf install -y git

# Amazon Linux 2 / 2023
sudo yum install -y git
```

Clone the repo and make the script executable:

```
git clone https://github.com/jham-s1/s1-data-pipeline-doctor.git
cd s1-data-pipeline-doctor
chmod +x data-pipeline-doctor.sh
```

NOTE

If Docker isn't already installed on this host, Data Pipeline Doctor detects the OS and offers to install Docker Engine for you (Amazon Linux 2/2023, RHEL/CentOS Stream/Rocky/AlmaLinux, Fedora, Ubuntu/Debian) the first time you run it. Fedora is detected and handled as its own OS family (separate from RHEL/Rocky/Alma) since it uses a different Docker CE repo path — you don't need to do anything differently, just know it's covered.

NOTE

After a fresh Docker Engine install, the installer adds your user to the `docker` group — but that only takes effect in a new shell session. If the wizard immediately re-checks and reports "Docker is installed but the daemon isn't reachable," don't troubleshoot Docker itself: just start a new session (re-SSH, open a new terminal, or run the tool once more with `sudo`) and re-run the command. This applies on every distro, not just RHEL.

4 Get the .env contents onto the deployment host

The file downloaded in Step 2 is **not** named `ai-data-pipelines.env` — the console names it something like `Observe_<site-id>_config.env`. That mismatch, plus the fact that Data Pipeline Doctor originally only looked for that fixed filename in a specific home directory,

turned out to be the single biggest source of "the wizard is stuck waiting" problems in practice: wrong home directory, wrong filename, the file landing in a `tmp` subdirectory instead, and even NFS/autofs-mounted home directories where the file was in the right place the whole time but a stale kernel attribute cache kept it invisible to the wizard's polling loop.

RECOMMENDED — DOCTOR V1.6+

Skip the file entirely. When you reach Step 5, if the wizard doesn't find `ai-data-pipelines.env` already in place, it now offers to paste the `.env` contents directly into the terminal instead of waiting on a file copy. This sidesteps every issue in this section — there's no file, no path, no home directory, and no cache to get wrong.

If you're going this route, skip straight to Step 5 and paste when prompted. Confirm you're on a current clone first: `git -C s1-data-pipeline-doctor log -1 --oneline` (the splash screen also prints the version).

Fallback: copy the file into place. Use this if you're on an older Doctor build, or you'd simply rather have the file on disk. Data Pipeline Doctor expects the fixed filename `ai-data-pipelines.env` in the home directory of whichever user account actually runs the script.

NFS / AUTOFS-MOUNTED HOME DIRECTORIES

If `$HOME` on this host is autofs/NFS-mounted (common in AD-integrated enterprise Linux — check with `mount | grep -i home`), the file-copy path can fail in a way that's genuinely confusing to debug: the file is placed correctly and `$HOME` resolves correctly, but the wizard's polling loop still never sees it. NFS clients cache directory attributes for a while (~30–60s by default) at the kernel/mount level — a negative lookup can stay cached well past when the file appears, and restarting the script doesn't help since the cache is shared across processes on that mount. Use the paste option above instead if this describes your environment.

If you're still using the file-copy path: `sudo` resets `$HOME` to `/root` by default. Doctor v1.6+ resolves this automatically when invoked via `sudo` — it detects `$SUDO_USER` and looks in that user's home directory, not `/root`. Only a direct root login/session (SSH'd in as root, or `su -`) still falls back to `/root`. If there's any chance this host gets administered that way, copy the file into both homes to be safe.

From your local machine, copy the downloaded file to the Linux host:

```
# macOS / Linux
scp /path/to/Observe*_config.env <user>@<linux-host>:~/downloaded.env
```

```
# Windows PowerShell – requires the OpenSSH Client optional feature
# (built into Windows 10 1809+/11)
scp C:\path\to\Observe_<site-id>_config.env <user>@<linux-host>:~/downloaded.env
```

NOTE

PowerShell doesn't expand `*` wildcards for external commands the way bash does — `scp`'s source argument needs the exact filename. Resolve the wildcard yourself first if preferred:

```
scp (Get-ChildItem "C:\path\to\Observe*_config.env").FullName `
  <user>@<linux-host>:~/downloaded.env
```

On the Linux host, rename it into place for your login user and for root, and lock down permissions on both copies (the file contains a registry auth token):

```
# Your login user's home directory
cp ~/downloaded.env ~/ai-data-pipelines.env
chmod 600 ~/ai-data-pipelines.env

# Also root's home directory, in case this host is ever administered via
# a direct root login/session rather than sudo (see the note above)
sudo cp ~/downloaded.env /root/ai-data-pipelines.env
sudo chmod 600 /root/ai-data-pipelines.env
```

5 Run Data Pipeline Doctor to deploy the site

From the cloned repo directory, run the tool and select the Docker site type:

```
./data-pipeline-doctor.sh --site-type docker
```

The tool opens with a splash screen — press **Enter** at the "Press Enter to begin..." prompt to continue. Since no `observe-standalone-site` container exists yet, the tool then starts its **New Site Setup** wizard, which:

1. Looks for `~/ai-data-pipelines.env`. If already copied into place, it's found immediately. Otherwise the wizard offers to let you paste the `.env` contents directly (Doctor v1.6+) — finish with `Ctrl-D` or a line containing just `EOF`. Either way the result is `chmod 600` 'd automatically.
2. Parses the embedded ECR registry, auth token, and image URI from the `.env` file and runs `docker login` against the registry automatically.
3. Optionally prompts to publish source listener ports (Syslog TCP/TLS/UDP, HEC, Kafka, or a custom mapping) — this can also be done later via the main menu's **Source ports** option.
4. Asks whether to manage the site with **Docker Compose** (recommended — generates `~/docker-compose-observo.yml`) or a plain `docker run`.
5. Starts the `observo-standalone-site` container using the `.env` file for configuration, with a named `observo-data` volume for persistent state.

IF YOU CHOOSE DOCKER COMPOSE

The generated `~/docker-compose-observo.yml` references the env file as `env_file: ~/ai-data-pipelines.env` — the `~` is expanded by whichever user's shell runs `docker compose`, not baked in as an absolute path. Manage the site (`up` / `down` / `restart`) as the same user that ran the setup wizard — otherwise Compose reads whichever env copy matches the invoking user's `$HOME`. Plain `docker run` doesn't have this ambiguity — the script expands the path itself.

Once the container is running, re-launch the same command (`./data-pipeline-doctor.sh -site-type docker`) any time to reach the full Docker diagnostic menu — full scan, service health, TLS/DNS checks, logs, proxy/connectivity tests, and more.

Verification

Confirm the site actually came up healthy and is receiving data before handing it off.

1. **Container is running:**

```
docker ps --filter name=observo-standalone-site
```

Status should be `Up`, not `Restarting` or `Exited`.

2. **s6 services are all up.** From the Doctor menu, option **2 — Service health (s6)** should show `control-plane-init`, `dataplane`, `control-agent`, `collector`, `pattern-extractor`, and `monitor` all healthy. If `control-plane-init` hasn't succeeded yet, the other five stay down — give it a minute after first start and re-check.
3. **Run a full diagnostic scan.** Option **1 — Full diagnostic scan** runs every Docker check (service health, container/resources, TLS, DNS, volume/disk, proxy/connectivity) in one pass and will flag and offer to fix anything wrong.
4. **Site shows Online in the console.** Back in **Data Pipelines → Pipeline Manager → Settings → Sites**, the site created in Step 1 should show status **Online**. If it's stuck Offline, see Troubleshooting below.
5. **Send a test event and confirm it lands in SDL.** Option **13 — Syslog tester** sends a synthetic event (Firewall, Auth, DNS, Web, or Generic) to a published source port on `localhost`, embedding a unique UUID. Search Singularity Data Lake for `message contains '<uuid>'` to confirm end-to-end delivery. Requires a source already configured in the console with a published container port (see **Source ports**, option 12).

Troubleshooting

Start with option **1 — Full diagnostic scan** — it runs every check below in one pass and offers fixes automatically where possible. The rest of this section maps specific symptoms to the right menu option.

Symptom	Likely cause	Doctor menu option
Site shows Offline in the console	Container not running, or can't reach the Manager	2 Service health · 3 Container & resources · 11 Proxy & connectivity
control-plane-init failing / all 5 services down	Connectivity or TLS to the control plane	1 Full scan · 11 Proxy & connectivity (auto-classifies, then offers a fix)
Data not flowing to AI SIEM	Logs-ingestion egress blocked (firewall/proxy)	11 Proxy & connectivity (tests the -logs/-metrics/-api endpoints specifically)
TLS handshake failures	CA not trusted / cert mismatch	4 TLS & certificates — shows the required recreate change
Connection refused on a source port	Port not published, or host firewall blocking it	12 Source ports — shows mappings, opens new ones, prints firewall hints
Events dropped	Disk queue full / memory exhausted	6 Volume & disk · 3 Container & resources
Container OOMKilled	Memory limit too low for throughput	3 Container & resources — bumps the limit live via docker update --memory
High CPU	Pipeline too complex, or throughput over sizing	3 Container & resources (limits vs. live docker stats)
Setup wizard stuck waiting for ~/ai-data-pipelines.env despite the file being there	Wrong home directory, wrong filename, or NFS/autofs stale attribute cache	Ctrl-C and re-run with the paste option — otherwise find ~ /tmp -iname 'ai-data-pipelines.env'
docker compose up fails with "env file ... not found"	Compose was run by a different user/\$HOME than expected	Copy the env file into that user's home too, or always invoke Compose as the setup user
A single s6 service down , container otherwise healthy	One service crashed independently	8 Service debugger → per-service logs, then 9 Restart service/container

Docker-specific auto-fixes applied without a recreate

Issue	Fix
control-plane-init retry loop	Classifies connectivity/TLS/auth, routes to the matching remedy, then offers a restart
Connectivity failure (EOF / refused / timeout)	Shows the required egress and exact docker run -e HTTPS_PROXY=... change, then offers a restart
TLS CA trust (certificate signed by unknown authority)	Shows the required -v <ca-bundle>:...:ro + SSL_CERT_FILE recreate change, then offers a restart
OOMKilled	Bumps the memory limit live with docker update --memory (falls back to a recreate command)
A single s6 service down	Restarts just that service (s6-svc) or the whole container

A running container's mounts and environment can't be changed in place — TLS and proxy fixes print the exact recreate command and offer to restart the container for you; they don't silently recreate it behind your back. If a fix requires recreating the container (new mount, new env var, new published port), the tool always renames the old container as a backup first and prints a one-line rollback, so a bad recreate is reversible:

```
docker rm -f observo-standalone-site
docker rename observo-standalone-site_old observo-standalone-site
docker start observo-standalone-site
```